PROCESS AND THREAD SECURITY

Operating System Goals

- Reliability
- Security
- Privacy
- Fair resource allocation

Protection is the job of the operating system kernel

Operating system problems

- Administrators have no way to control users
  - Administrators have unlimited control
- Processes can change security properties
- Processes inherit user rights

Security philosophy

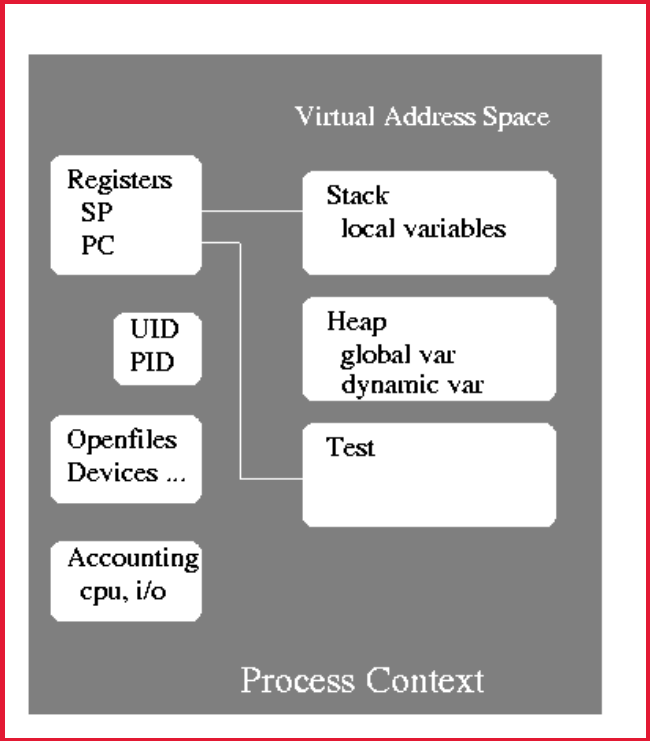
- Multi-user system security objective should be to isolate user resources from one another
  - Prevent user A from reading user B's files
  - Prevent user A from exhausting user B's memory
  - Prevent user A from exhausting user B's CPU resources
  - Prevent user A from exhausting user B's devices

Processes – Defined

- The process is the OS's abstraction for execution
  - the unit of execution
  - a unit of scheduling
  - the dynamic execution context
- Process is often called a job, task, or sequential process

Process Model

- Process: a program in execution
  - IE to web browsing, Outlook to read email, Word to write reports
- A process is a full blown virtual machine, defining its own address space and maintains running state
  - Address space: the set of memory locations that can be generated and accessed directly by a program; enforced by hw for protection
  - I/O part of the machine is accessed through OS calls.

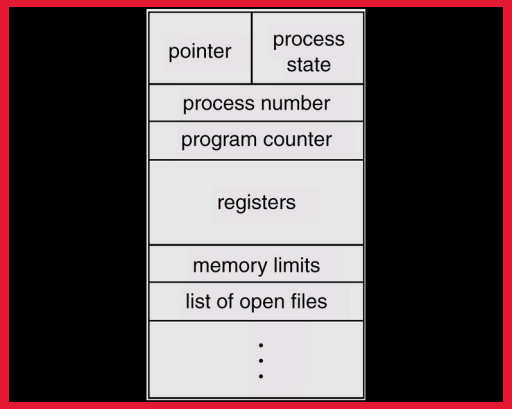


Processes in the OS – PCB

- Typical PCB contains:
  - execution state
  - PC, SP & processor registers – stored when process is made inactive
  - memory management info
  - Privileges and owner info
  - scheduling priority
  - resource info
  - accounting info

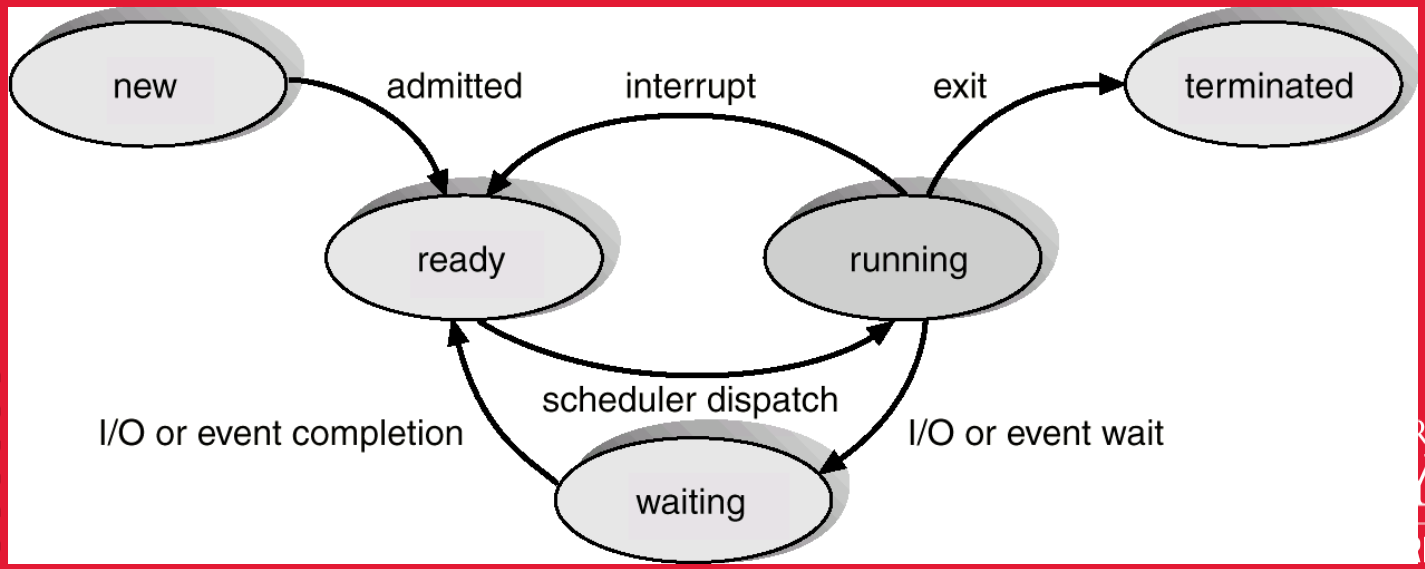
Under the Hood

- Process Control Block (PCB): a key data structure that maintains info associated with each process:
  - Process state
  - Program counter
  - CPU registers
  - CPU scheduling information
  - Memory-management info
  - Accounting information
  - I/O status information

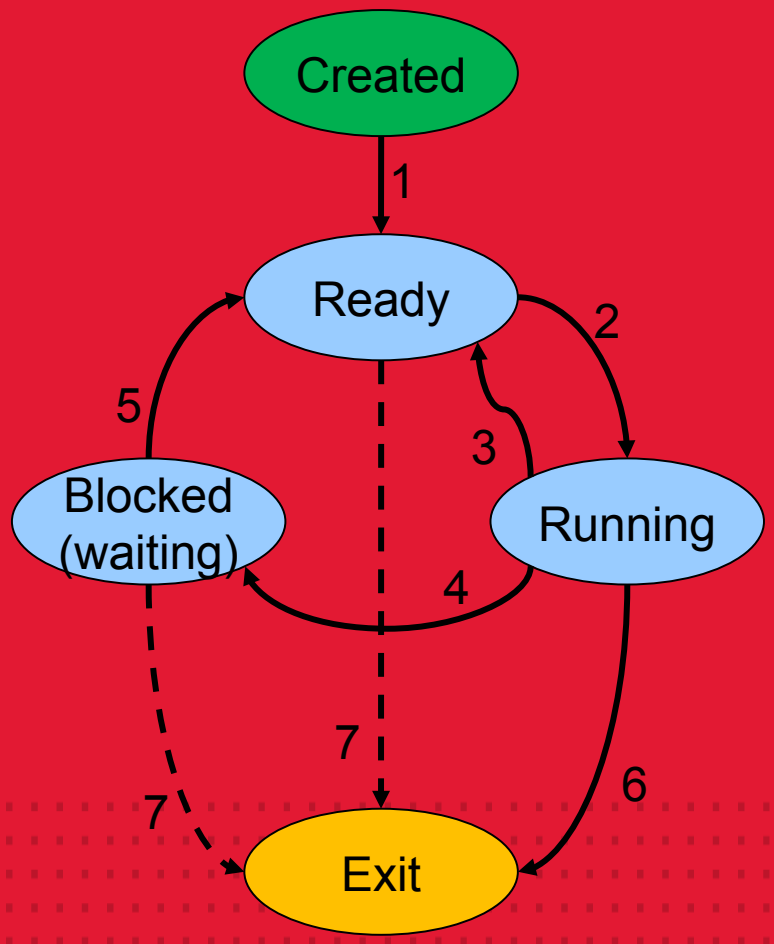


Process State

- As a process executes, it changes *state*
  - new: The process is being created.
  - running: Instructions are being executed.
  - waiting: The process is waiting for some event to occur.
  - ready: The process is waiting to be assigned to a process.
  - terminated: The process has finished execution.



Process states



- Process in one of 5 states
  - Created
  - Ready
  - Running
  - Blocked
  - Exit
- Transitions between states

  1 - Process enters ready queue

  2 - Scheduler picks this process

  3 - Scheduler picks a different process

  4 - Process waits for event (such as I/O)

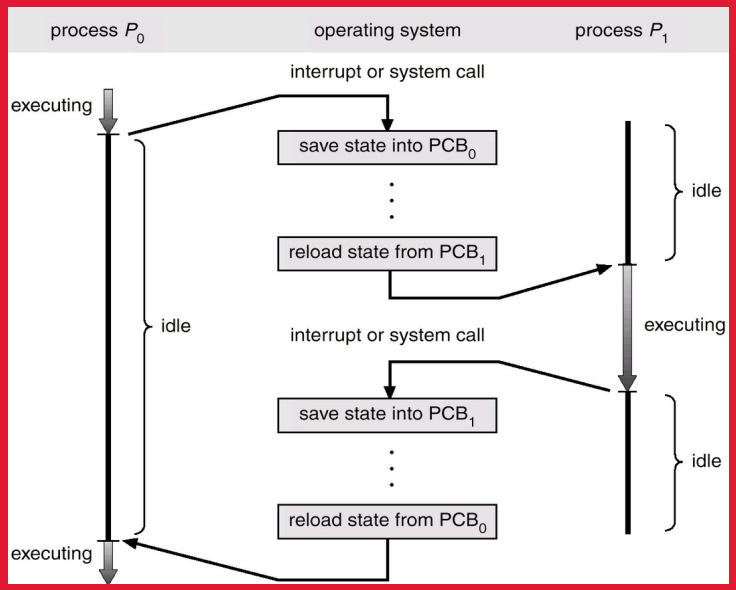
  5 - Event occurs

  6 - Process exits

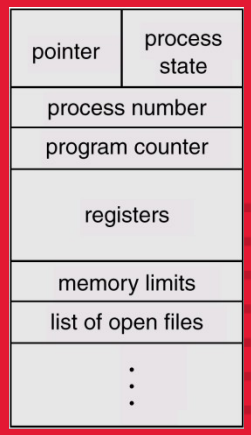
  7 - Process ended by another process

Context Switch

- When CPU switches to another process, the system must save the state of the old process and load the saved state for the new process.
- Context-switch time is overhead; the system does no useful work while switching.
- Time dependent on hardware support.

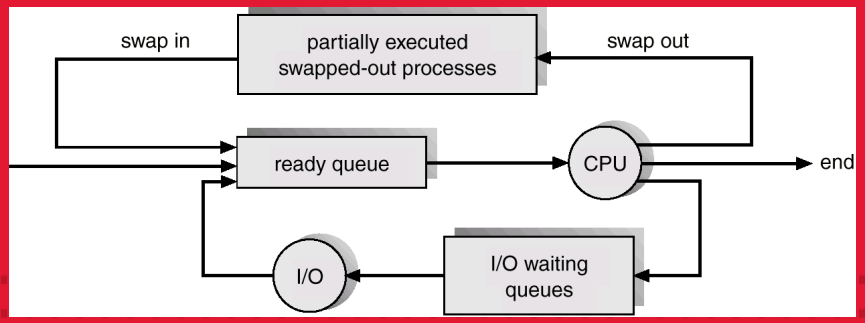


PCB



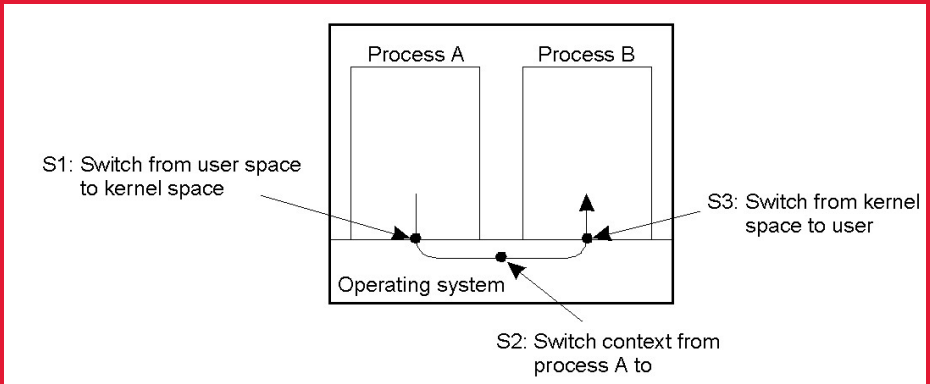
Process Schedulers

- Long-term scheduler (or job scheduler) – selects which processes should be brought into the ready queue.
  - Invoke infrequently (seconds, minutes)
  - Control the degree of concurrency
- Short-term scheduler (or CPU scheduler) – selects which process should be executed next and allocates CPU.
  - Invoke frequently (millisecond) enough to provide a perception of concurrent execution on single core system; must be fast
  - Optimize throughput, average response time, slowdown, revenue, etc
  - Slowdown is a normalized queuing delay w.r.t. exec time
  - On multiprocessor or multi-core system, real concurrent execution



Interprocess Comm (IPC)

- Processes are usually communicate by sending msgs back and forth via the OS



- Processes can share a segment of physical memory by mapping it to their address spaces
  - data in the segment can be accessed by processes for faster IPC

Thread and Multithreaded Proc

- A thread of control is a seq of instructions being executed within a process context. Each has its own logical control flow (pc)
- A multithreaded process has two or more threads within the same context. They share code, data in heap, and kernel context(open files, timers, etc)
- Each thread has its own ID

Thread 1 (main thread)

stack 1

Thread 1 context:
- Data registers
- Condition codes
- SP1
- PC1

Shared code and data

shared libraries

run-time heap

read/write data

read-only code/data

0

Kernel context:
- VM structures
- Open files
- Signal handlers
- brk pointer

Thread 2 (peer thread)

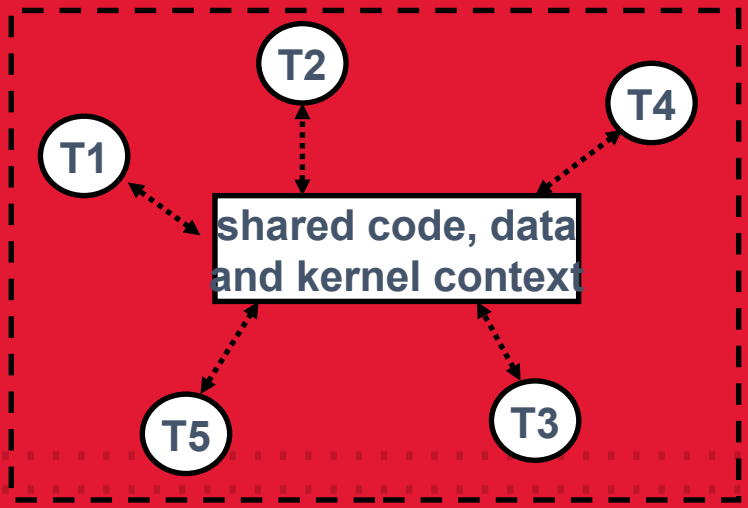
stack 2

Thread 2 context:
- Data registers
- Condition codes
- SP2
- PC2

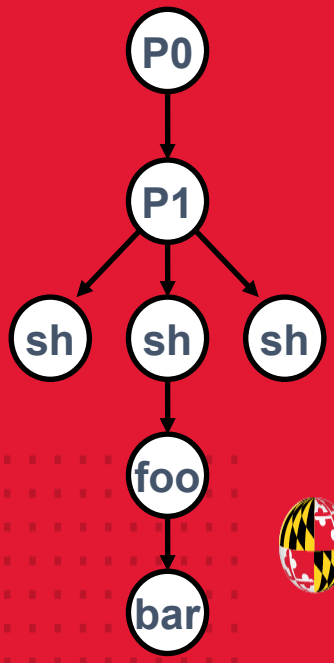
Logical View of Threads

- Threads associated with a process form a pool of peers.
  - unlike processes which form a tree hierarchy

Threads associated with process foo



Process hierarchy



Process vs Threads

- Processes are typically independent, while threads exist as subsets of a process
- Processes carry considerable state info, whereas multiple threads within a process share state as well as memory and other resources
- Processes have separate address space, whereas threads share their address space
- Processes interact through system-provided IPC
- Context switching between threads is an order of magnitude faster than context switching between processes

Why Multithreading

- Improve program structure
  - e.g. producer-consumer problem
- Efficiency
  - Creating a process calls into OS, which duplicates entire address space
  - Synchronizing processes need to trap into the OS, too.
  - Threads could be created in user-space
  - Threads could be synchronized by monitoring shared variables
- Match multi-core and multiprocessor arch
- Improve application responsiveness
  - overlapping I/O
    - e.g. Multithreaded clients, fast file read/write
  - asynchronous event handling
    - e.g. network-centric server , GUI

Processes – Contents

- A process in Unix or Windows comprises (at least):
  - an address space – usually protected and virtual – mapped into memory
  - the code for the running program
  - the data for the running program
  - an execution stack and stack pointer (SP)
  - the program counter (PC)
  - a set of processor registers – general purpose and status
  - a set of system resources
    - files, network connections, privileges, …

Processes

- Execution of the program with restricted rights
- Must have permission from the kernel before
  - Accessing memory of another process
  - Reading/writing to disk
  - Changing hardware setting
- Process is an *instance of a program*

Processes – Defined

- The process is the OS's abstraction for execution
  - the unit of execution
  - a unit of scheduling
  - the dynamic execution context
- Process is often called a job, task, or sequential process

Representation

- To users (and other processes) a process is identified by its *Process ID* (PID)
- In the OS, processes are represented by entries in a *Process Table* (PT)
  - PID "points to" a PT entry
  - PT entry = *Process Control Block* (PCB)
- PCB is a large data structure that contains or points to all info about the process
  - Linux - defined in **task_struct** see `include/linux/sched.h` – over 70 fields
  - NT – defined in **EPROCESS** – about 60 fields

Process – starting and ending

- Process start
  - When the system boots
  - By the actions of another process
  - By the actions of a user
  - By the actions of a batch manager
- Processes end
  - Normally – exit
  - Voluntarily on an error
  - Involuntarily on an error
  - Terminated (killed) by the actions a user or a process

Processes – Switching

- When a process is running, its hardware state is in the CPU – PC, SP, processor registers
- When the OS suspends running a process, it saves the hardware state in the PCB
- *Context switch* is the act of switching the CPU from one process to another
  - timesharing systems may do 100s or 1000s of switches/sec
  - takes 1-100 microseconds on today's hardware

Processes – State Queues

- The OS maintains a collection of *process state* queues
  - typically one queue for each state – e.g., ready, waiting, …
  - each PCB is put onto a state queue according to its current state
  - as a process changes state, its PCB is unlinked from one queue, and linked to another
- Process state and the queues change in response to events – interrupts, traps

Processes – Privileges

This is a brief introduction to a facet of system security which will be covered in more detail in the future

- Users are given privileges by the system administrator
  - Privileges determine what **rights** a user has for an **object**.
    - Unix/Linux - Read|Write|eXecute by user, group and "world"
    - Windows – **A**ccess **C**ontrol **L**ist
  - Processes "inherit" privileges from user

Processes – Creation

- Unix/Linux
  - Create a new (child) process – fork();
    - Allocates new PCB
    - Clones the calling process (almost)
      - Copy of parent process address space
      - Copies resources in kernel (e.g. files)
    - Places PCB on <u>Ready</u> queue
    - Return from call
      - 0 for child
      - child PID for parent

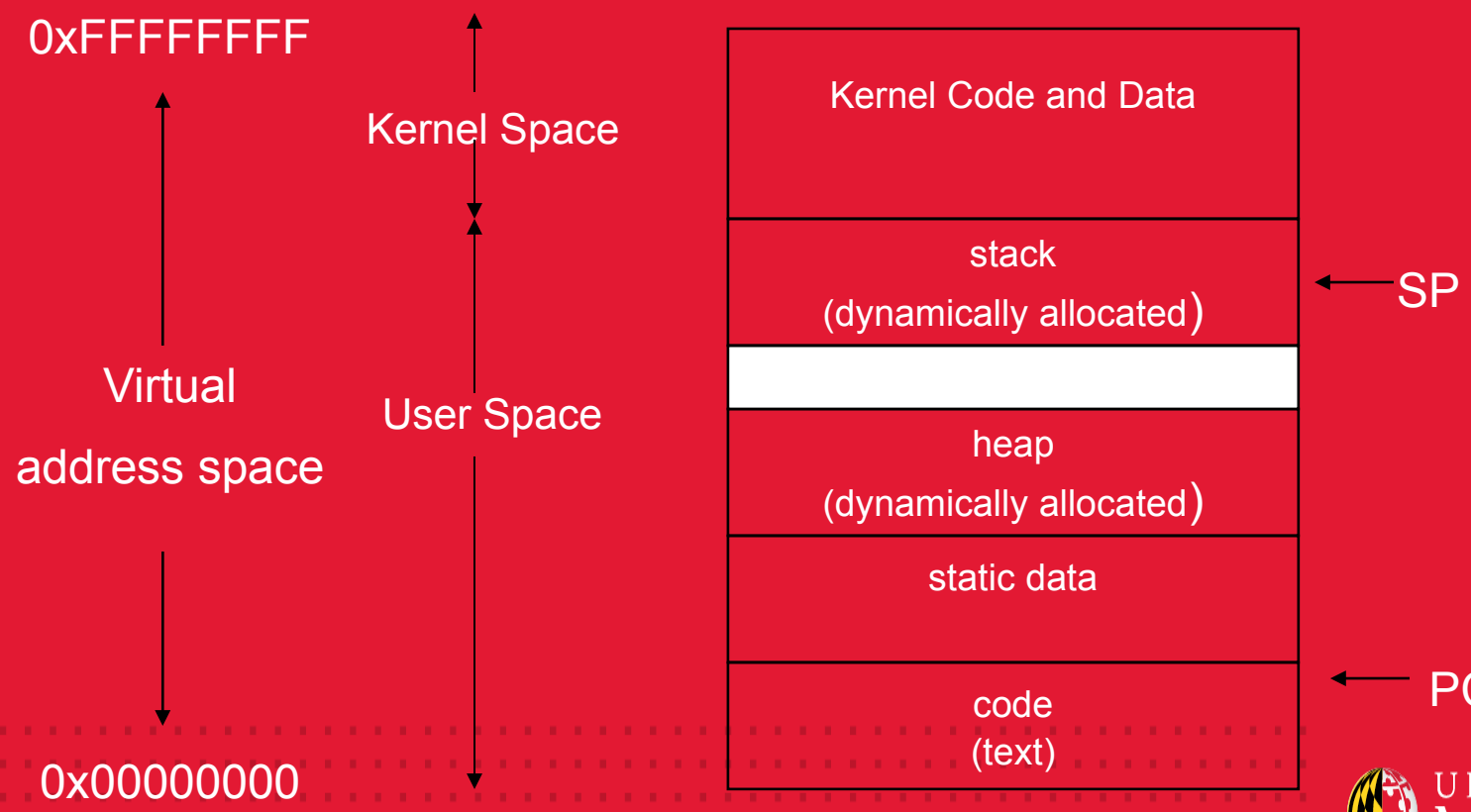
fork( )

```
int main(int argc, char **argv)
{
  char *name = argv[0];
  int child_pid = fork();
  if (child_pid == 0) {
    printf("Child of %s is %d\n",
           name, child_pid);
    return 0;
  } else {
    printf("My child is %d\n", child_pid);
    return 0;
  }
}
```

Processes – New Programs

- Starting another program
  - Unix – int exec (char *prog, char **argv)
    - Check privileges and file type
    - Loads program “prog” into address space
    - Initializes context – e.g. passes arguments (*argv)
    - Place PCB on ready queue
  - Windows – combines fork & exec
    - CreateProcess (`10 arguments`)
    - Not a parent child relationship
    - Note – privileges required to create a new process

Processes – Address Space



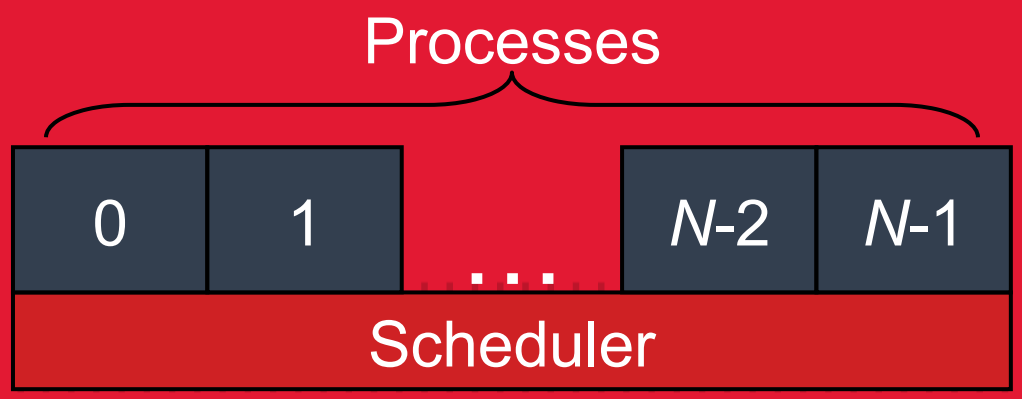
Linux – 3G/1G Windows – 2G/2G user space/kernel space

Exec – shell-like

```
int main(int argc, char **argv)
{ char *argvNew[5];
  int pid;
  if ((pid = fork()) < 0) {
          printf( "Fork error\n");
          exit(1);
  } else if (pid == 0) { /* child process */
          argvNew[0] = "/bin/ls";
          argvNew[1] = "-l";
          argvNew[2] = NULL;
          if (execve(argvNew[0], argvNew, environ) < 0) {
             printf( "Execve error\n");
             exit(1);
          }
    } else { /* parent */
```

Processes in the OS

- Two “layers” for processes
- Lowest layer of process-structured OS handles interrupts, scheduling
- Above that layer are sequential processes
  - Processes tracked in the *process table*
  - Each process has a *process table entry*



What's in a process's table entry?

May be stored on stack

Process management
- Registers
- Program counter
- CPU status word
- Stack pointer
- Process state
- Priority / scheduling parameters
- Process ID
- Parent process ID
- Signals
- Process start time
- Total CPU usage

File management
- Root directory
- Working (current) directory
- File descriptors
- User ID
- Group ID

Memory management
Pointers to text, data, stack
or
Pointer to page table

What Happens on a Trap/Interrupt?

1. Hardware saves program counter (on stack or in a special register)
2. Hardware loads new PC, identifies interrupt
3. Assembly language routine saves registers
4. Assembly language routine sets up stack
5. Assembly language calls C to run service routine
6. Service routine calls scheduler
7. Scheduler selects a process to run next (might be the one interrupted…)
8. Assembly language routine loads PC & registers for the selected process

Process Management

- UNIX process management separates the **creation** of processes and the **running** of a new program into two distinct operations.
  - The **fork** system call creates a new process
  - A new program is run after a call to **execve**
- Under UNIX, a process encompasses all the information that the operating system must maintain to track the context of a single execution of a single program
- Under Linux, process properties fall into three groups: the process's identity, environment, and context

Process Identity

- Process ID (PID). The unique identifier for the process; used to specify processes to the operating system when an application makes a system call to signal, modify, or wait for another process
- Credentials. Each process must have an associated user ID and one or more group IDs that determine the process's rights to access system resources and files
- Personality. Not traditionally found on UNIX systems, but under Linux each process has an associated personality identifier that can slightly modify the semantics of certain system calls
  - Used primarily by emulation libraries to request that system calls be compatible with certain specific flavors of UNIX

Process Environment

- The process's environment is inherited from its parent, and is composed of two null-terminated vectors:
  - The argument vector lists the command-line arguments used to invoke the running program; conventionally starts with the name of the program itself
  - The environment vector is a list of "NAME=VALUE" pairs that associates named environment variables with arbitrary textual values
- Passing environment variables among processes and inheriting variables by a process's children are flexible means of passing information to components of the user-mode system software
- The environment-variable mechanism provides a customization of the operating system that can be set on a per-process basis, rather than being configured for the system as a whole

Process Context

- The (constantly changing) state of a running program at any point in time
- The **scheduling context** is the most important part of the process context; it is the information that the scheduler needs to suspend and restart the process
- The kernel maintains **accounting** information about the resources currently being consumed by each process, and the total resources consumed by the process in its lifetime so far
- The **file table** is an array of pointers to kernel file structures
  - When making file I/O system calls, processes refer to files by their index into this table

Process Context (Cont.)

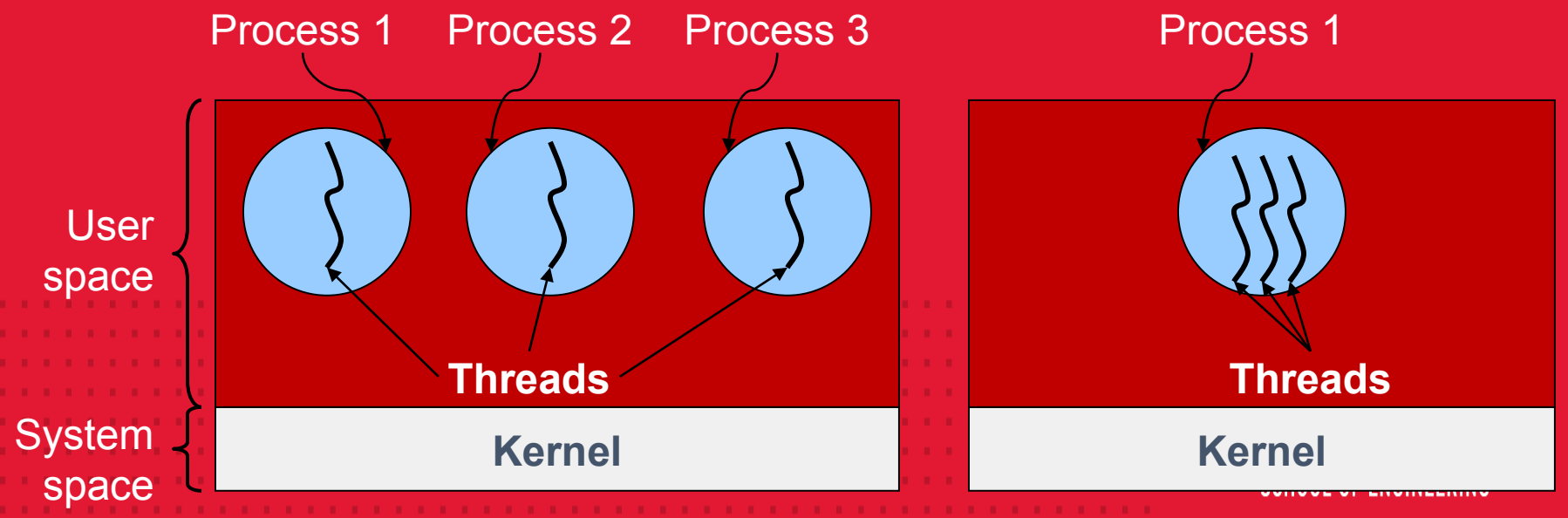
- Whereas the file table lists the existing open files, the **file-system context** applies to requests to open new files
  - The current root and default directories to be used for new file searches are stored here
- The **signal-handler table** defines the routine in the process's address space to be called when specific signals arrive
- The **virtual-memory context** of a process describes the full contents of the its private address space

Processes and Threads

- Linux uses the same internal representation for processes and threads; a thread is simply a new process that happens to share the same address space as its parent
- A distinction is only made when a new thread is created by the **clone** system call
  - **fork** creates a new process with its own entirely new process context
  - **clone** creates a new process with its own identity, but that is allowed to share the data structures of its parent
- Using **clone** gives an application fine-grained control over exactly what is shared between two threads

Threads: processes sharing memory

- Process == address space
- Thread == program counter / stream of instructions
- Two examples
  - Three processes, each with one thread
  - One process with three threads



Processes and thread information

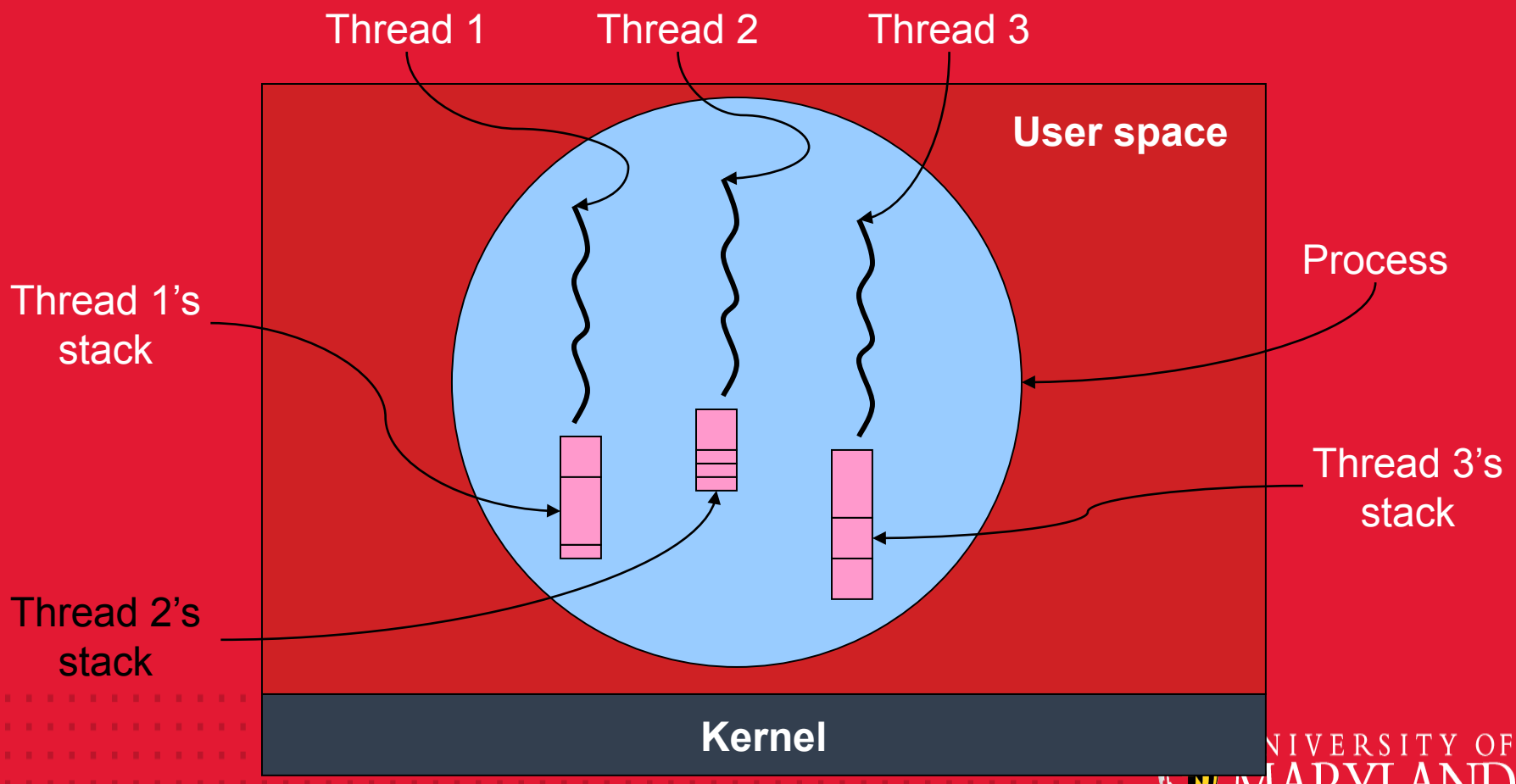
Per process items
Address space
Open files
Child processes
Signals & handlers
Accounting info
Global variables

Per thread items
Program counter
Registers
Stack & stack pointer
State

Per thread items
Program counter
Registers
Stack & stack pointer
State

Per thread items
Program counter
Registers
Stack & stack pointer
State

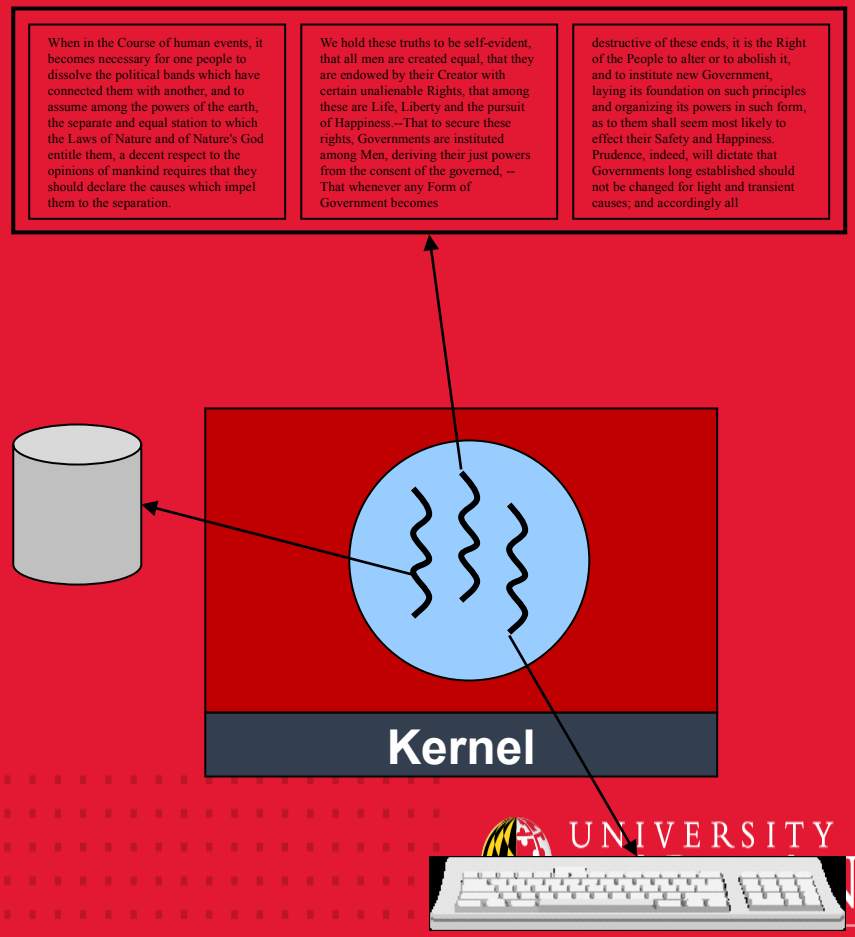
Threads and stacks



=> Each thread has its own stack!

Why use threads?

- Allow a single application to do many things at once
  - Simpler programming model
  - Less waiting
- Threads are faster to create or destroy
  - No separate address space
- Overlap computation and I/O
  - Could be done without threads, but it's harder
- Example: word processor
  - Thread to read from keyboard
  - Thread to format document
  - Thread to write to disk

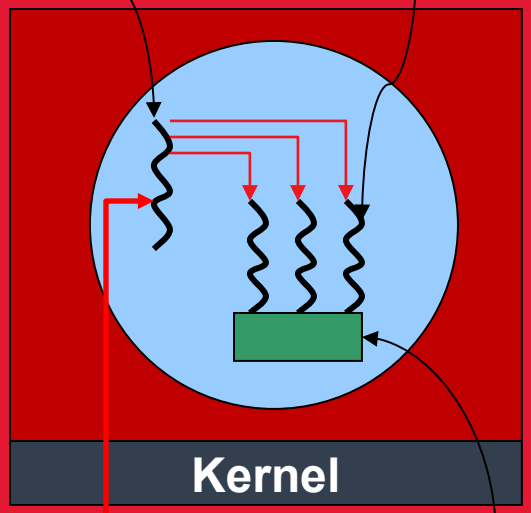


Multithreaded web server

Dispatcher thread

```
while(TRUE) {
  getNextRequest(&buf);
  handoffWork(&buf);
}
```

Worker thread



Web page cache

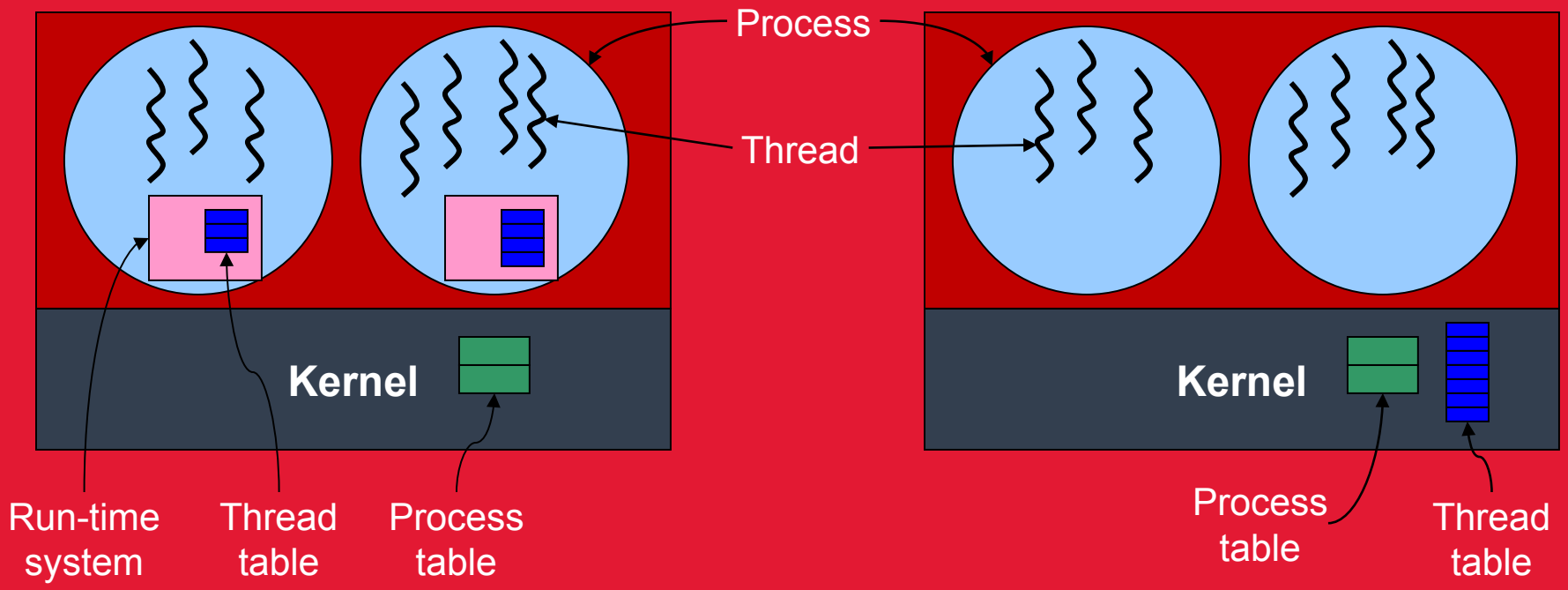
Network connection

```
while(TRUE) {
  waitForWork(&buf);
  lookForPageInCache(&buf,&page);
  if(pageNotInCache(&page)) {
    readPageFromDisk(&buf,&page);
  }
  returnPage(&page);
}
```

Three ways to build a server

- Thread model
  - Parallelism
  - Blocking system calls
- Single-threaded process: slow, but easier to do
  - No parallelism
  - Blocking system calls
- Finite-state machine
  - Each activity has its own state
  - States change when system calls complete or interrupts occur
  - Parallelism
  - Nonblocking system calls

Implementing threads



User-level threads
+ No need for kernel support
- May be slower than kernel threads
- Harder to do non-blocking I/O

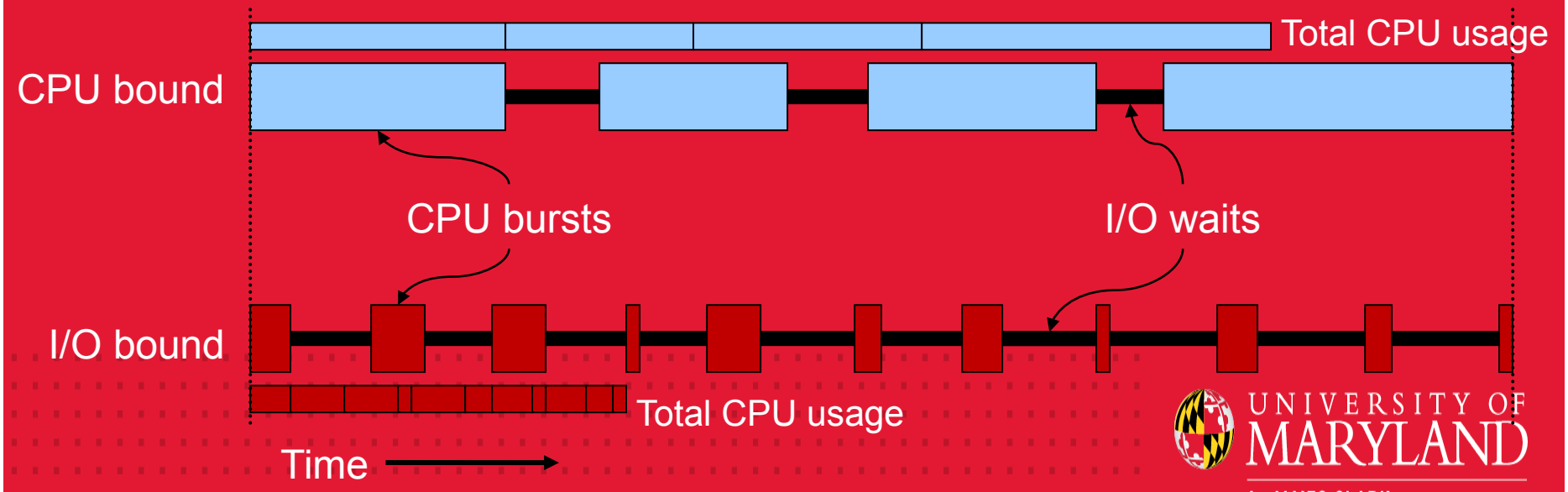
Kernel-level threads
+ More flexible scheduling
+ Non-blocking I/O
- Not portable

Scheduling

- What is scheduling?
  - Goals
  - Mechanisms
- Scheduling on batch systems
- Scheduling on interactive systems
- Other kinds of scheduling
  - Real-time scheduling

Why schedule processes?

- Bursts of CPU usage alternate with periods of I/O wait
- Some processes are *CPU-bound*: they don't make many I/O requests
- Other processes are *I/O-bound* and make many kernel requests



When are Processes Scheduled?

- At the time they enter the system
  - Common in batch systems
  - Two types of batch scheduling
    - Submission of a new job causes the scheduler to run
    - Scheduling only done when a job voluntarily gives up the CPU (*i.e.*, while waiting for an I/O request)
- At relatively fixed intervals (clock interrupts)
  - Necessary for interactive systems
  - May also be used for batch systems
  - Scheduling algorithms at each interrupt, and picks the next process from the pool of “ready” processes

Scheduling goals

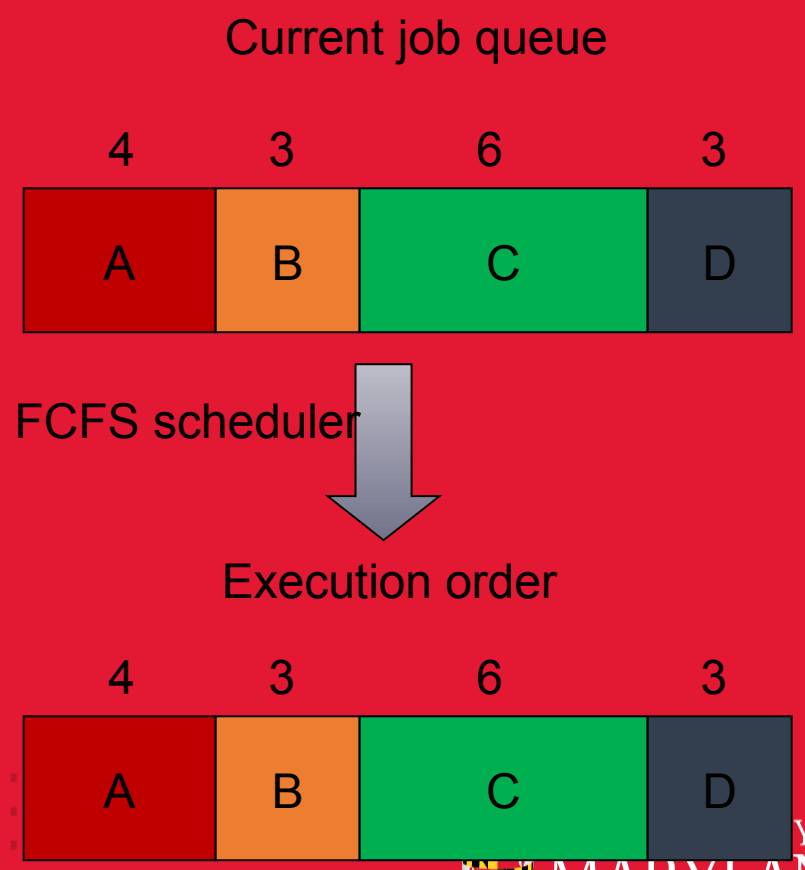
- All systems
  - Fairness: give each process a fair share of the CPU
  - Enforcement: ensure that the stated policy is carried out
  - Balance: keep all parts of the system busy
- Batch systems
  - Throughput: maximize jobs per unit time (hour)
  - Turnaround time: minimize time users wait for jobs
  - CPU utilization: keep the CPU as busy as possible
- Interactive systems
  - Response time: respond quickly to users' requests
  - Proportionality: meet users' expectations
- Real-time systems
  - Meet deadlines: missing deadlines is a system failure!
  - Predictability: same type of behavior for each time slice

Measuring Scheduling Performance

- Throughput
  - Amount of work completed per second (minute, hour)
  - Higher throughput usually means better utilized system
- Response time
  - Response time is time from when a command is submitted until results are returned
  - Can measure average, variance, minimum, maximum, …
  - May be more useful to measure time spent waiting
- Turnaround time
  - Like response time, but for batch jobs (response is the completion of the process)
- Usually not possible to optimize for *all* metrics with the same scheduling algorithm

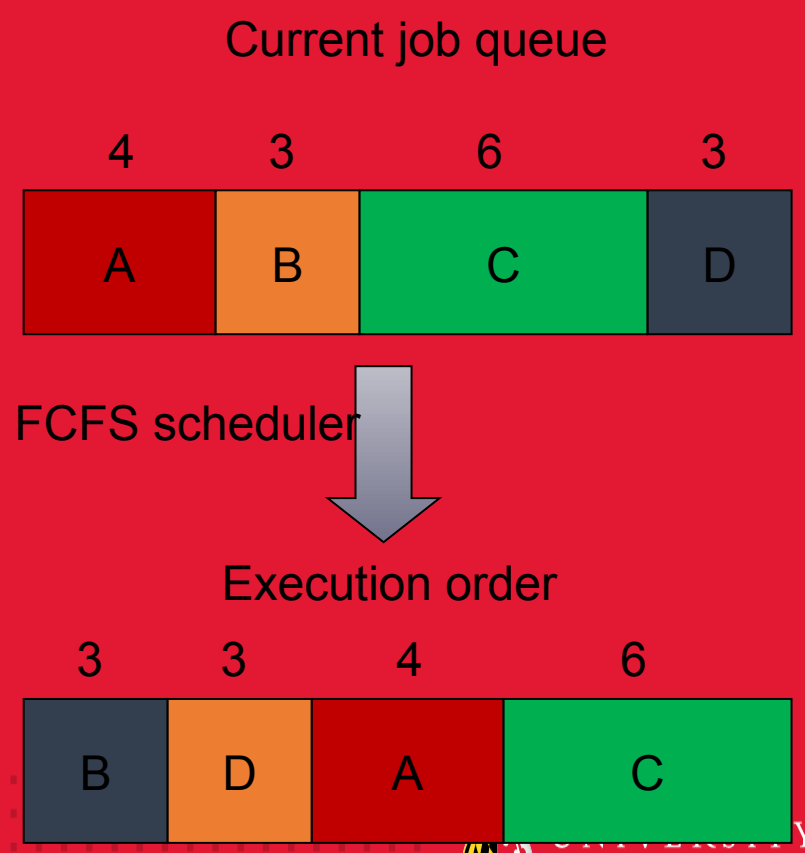
First Come, First Served (FCFS)

- Goal: do jobs in the order they arrive
  - Fair in the same way a bank teller line is fair
- Simple algorithm!
- Problem: long jobs delay every job after them
  - Many processes may wait for a single long job



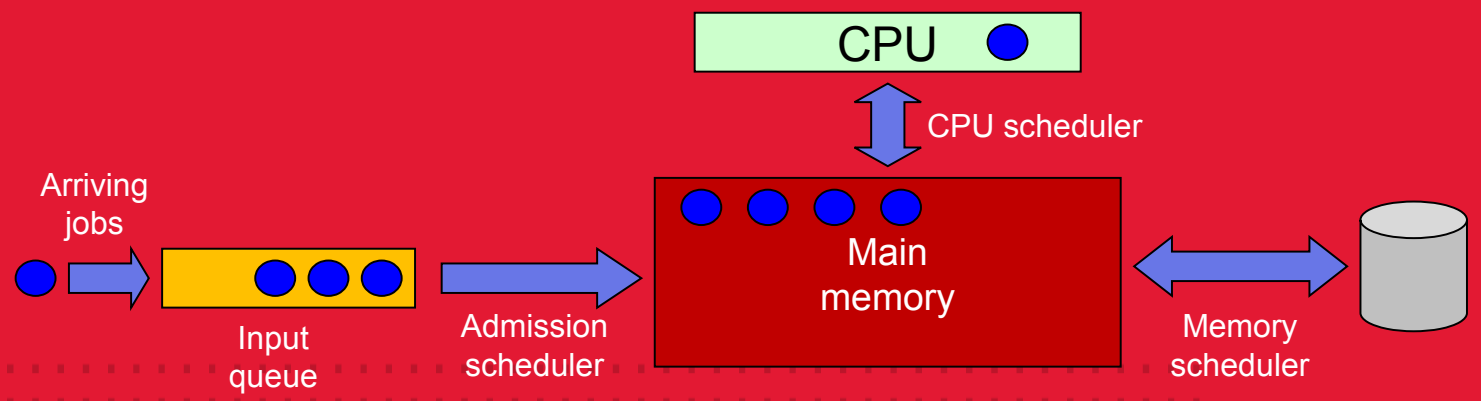
Shortest Job First (SJF)

- Goal: do the shortest job first
  - Short jobs complete first
  - Long jobs delay every job after them
- Jobs sorted in increasing order of execution time
  - Ordering of ties doesn't matter
- Shortest Remaining Time First (SRTF): preemptive form of SJF
- Problem: how does the scheduler know how long a job will take?



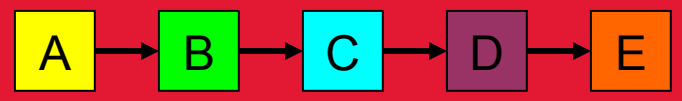
Three-Level Scheduling

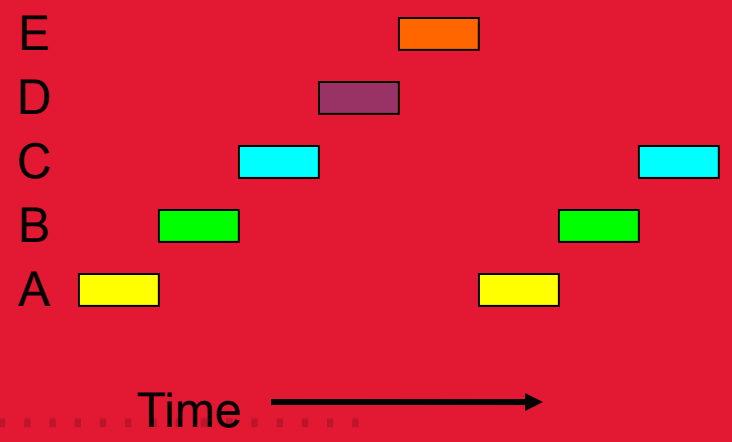
- Jobs held in input queue until moved into memory
  - Pick “complementary jobs”: small & large, CPU- & I/O-intensive
  - Jobs move into memory when admitted
- CPU scheduler picks next job to run
- Memory scheduler picks some jobs from main memory and moves them to disk if insufficient memory space



Round Robin (RR) Scheduling

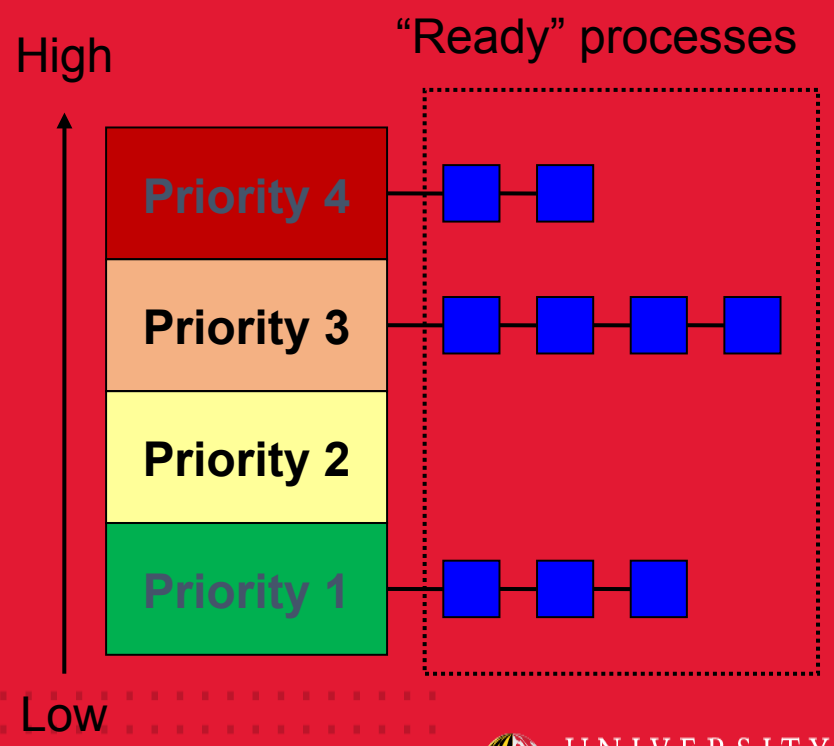
- Round Robin scheduling
  - Give each process a fixed time slot (*quantum*)
  - Rotate through "ready" processes
  - Each process makes some progress
- What's a good quantum?
  - Too short: many process switches hurt efficiency
  - Too long: poor response to interactive requests
  - Typical length: 10–50 ms





Priority Scheduling

- Assign a priority to each process
  - "Ready" process with highest priority allowed to run
  - Running process may be interrupted after its quantum expires
- Priorities may be assigned dynamically
  - Reduced when a process uses CPU time
  - Increased when a process waits for I/O
- Often, processes grouped into multiple queues based on priority, and run round-robin per queue



Shortest Process Next

- Run the process that will finish the soonest
  - In interactive systems, job completion time is unknown!
- Guess at completion time based on previous runs
  - Update estimate each time the job is run
  - Estimate is a combination of previous estimate and most recent run time
- Not often used because round robin with priority works so well!

Lottery scheduling

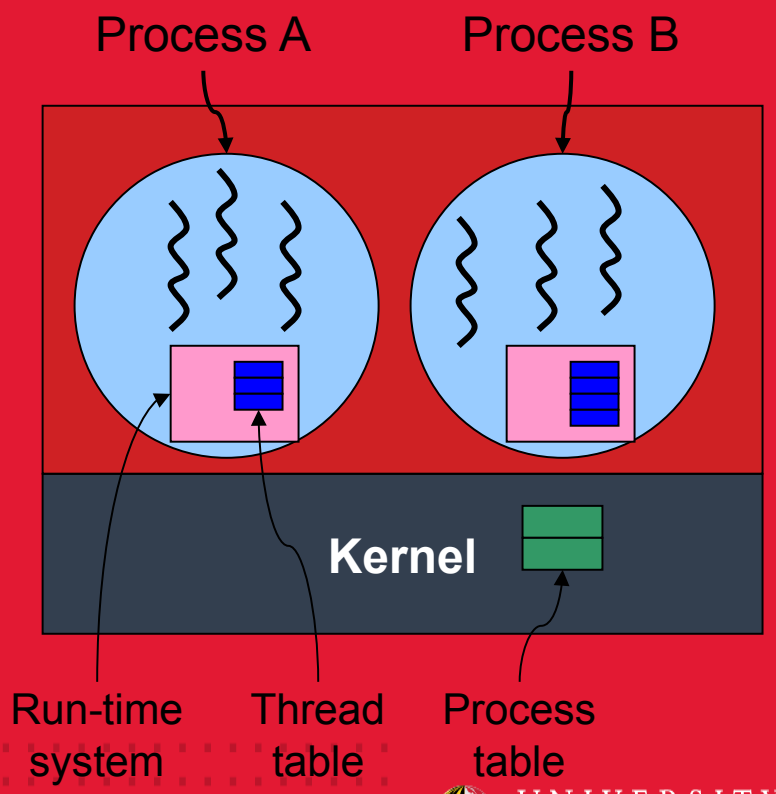
- Give processes “tickets” for CPU time
  - More tickets => higher share of CPU
- Each quantum, pick a ticket at random
  - If there are *n* tickets, pick a number from 1 to *n*
  - Process holding the ticket gets to run for a quantum
- Over the long run, each process gets the CPU *m*/*n* of the time if the process has *m* of the *n* existing tickets
- Tickets can be transferred
  - Cooperating processes can exchange tickets
  - Clients can transfer tickets to server so it can have a higher priority

Policy Versus Mechanism

- Separate what ***may*** be done from ***how*** it is done
  - Mechanism allows
    - Priorities to be assigned to processes
    - CPU to select processes with high priorities
  - Policy set by what priorities are assigned to processes
- Scheduling algorithm parameterized
  - Mechanism in the kernel
  - Priorities assigned in the kernel or by users
- Parameters may be set by user processes
  - Don't allow a user process to take over the system!
  - Allow a user process to voluntarily lower its own priority
  - Allow a user process to assign priority to its threads

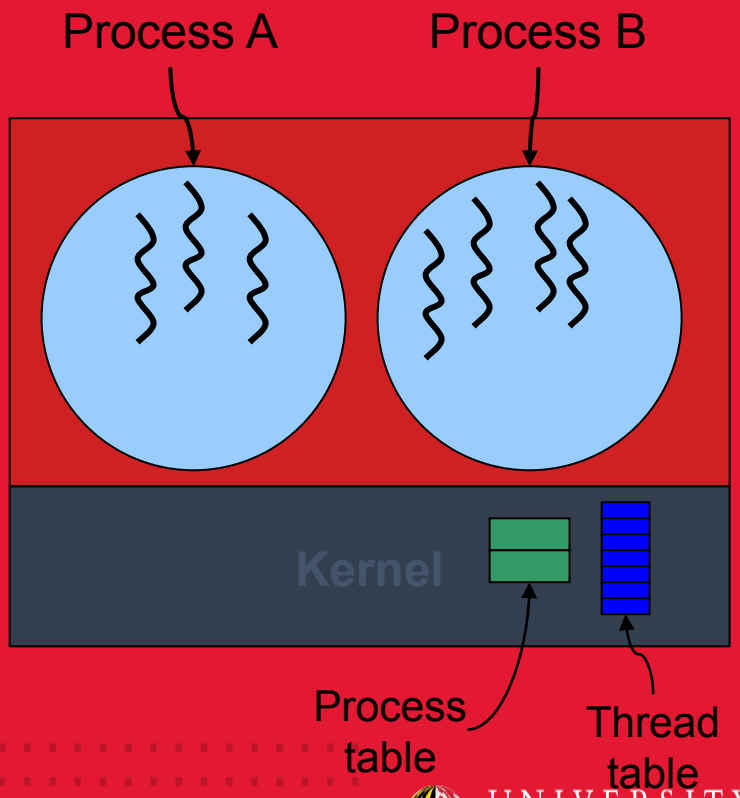
Scheduling User-Level Threads

- Kernel picks a process to run next
- Run-time system (at user level) schedules threads
  - Run each thread for less than process quantum
  - Example: processes get 40ms each, threads get 10ms each
- Example schedule: A1,A2,A3,A1,B1,B3,B2,B3
- Not possible: A1,A2,B1,B2,A3,B3,A2,B1

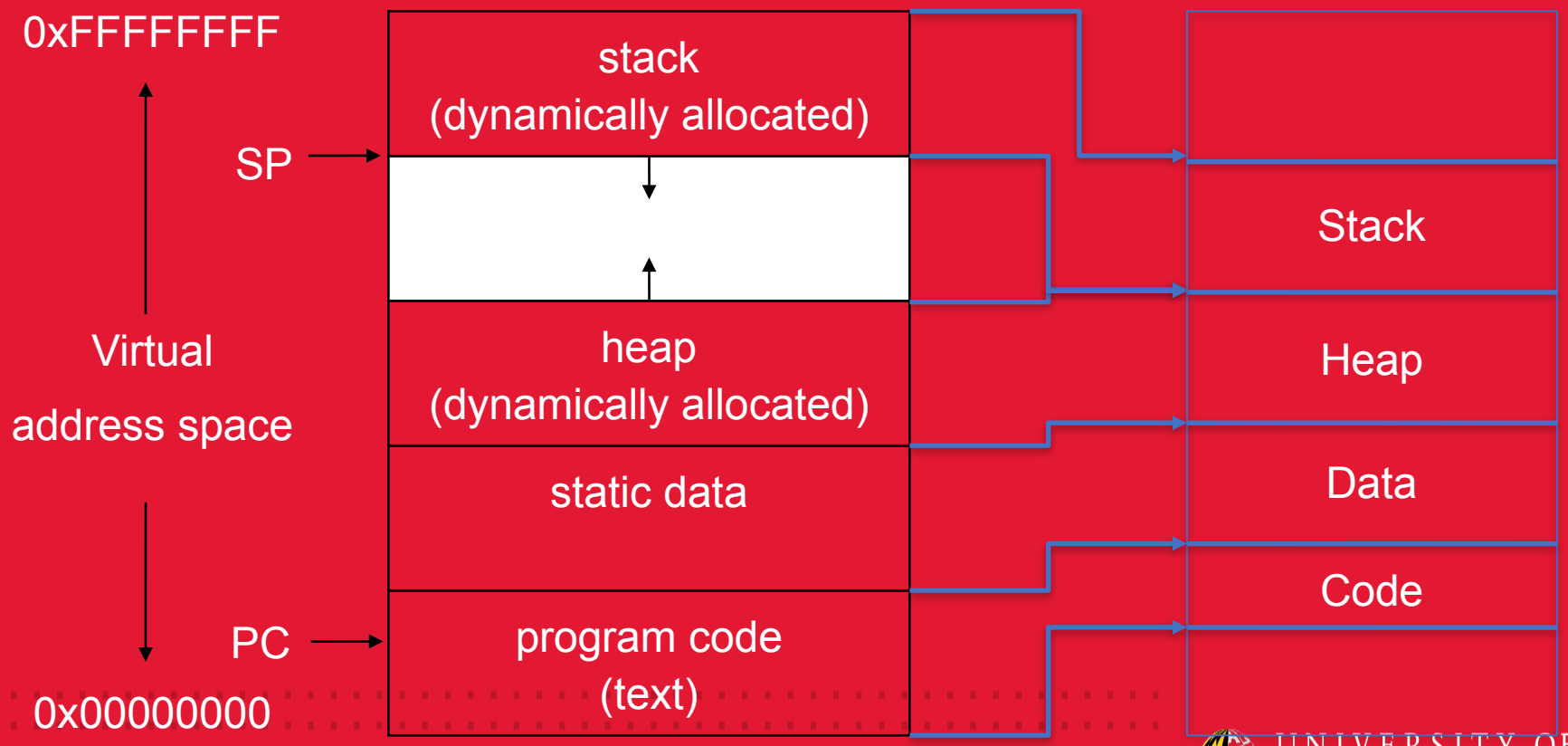


Scheduling Kernel-level threads

- Kernel schedules each thread
  - No restrictions on ordering
  - May be more difficult for each process to specify priorities
- Example schedule: A1,A2,A3,A1,B1,B3,B2,B3
- Also possible: A1,A2,B1,B2,A3,B3,A2,B1



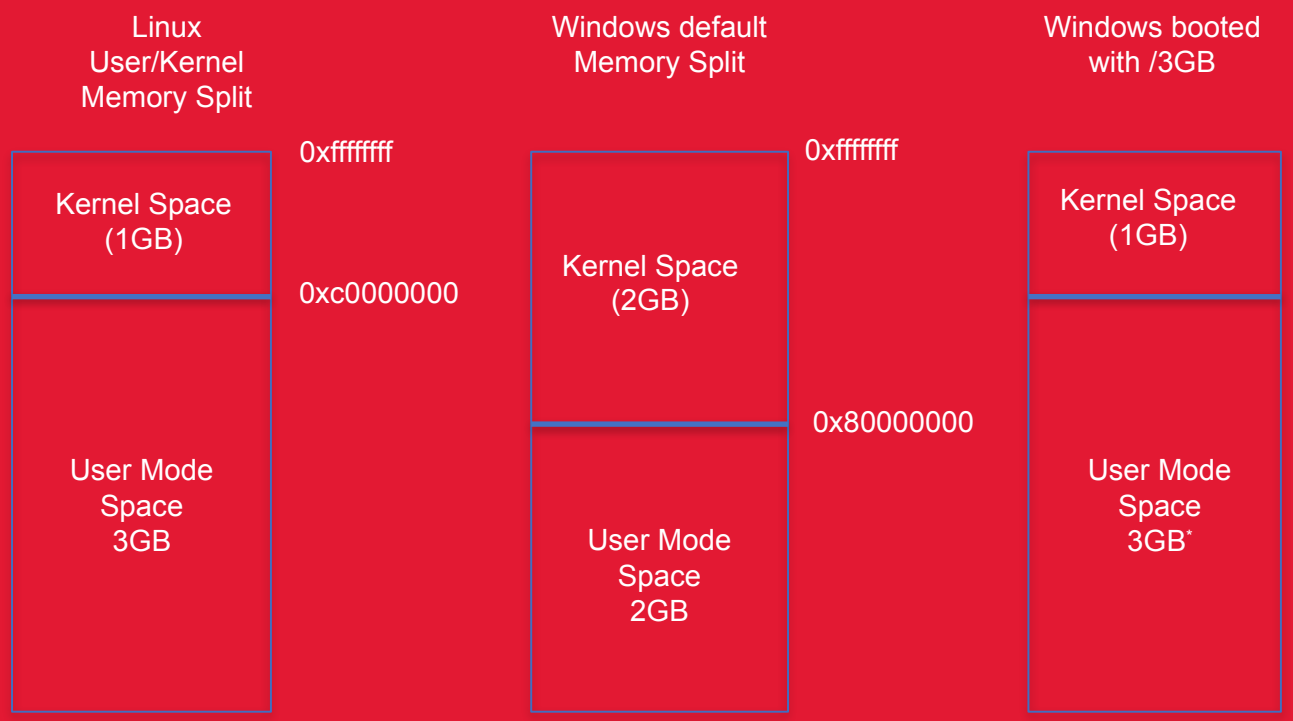
Process Address space



Virtual Address Space

- Every process's memory starts at zero
- Every process thinks that it has the machine all to itself
- VAS is mapped to physical addresses through page tables
  - Maintained by the kernel
  - Consulted by processor
- One caveat:
  - Virtual addresses apply to all software – including the kernel
- Looks like a sandbox
  - 32-bit mode: 4GB
  - 64-bit mode: 8TB

Linux and windows

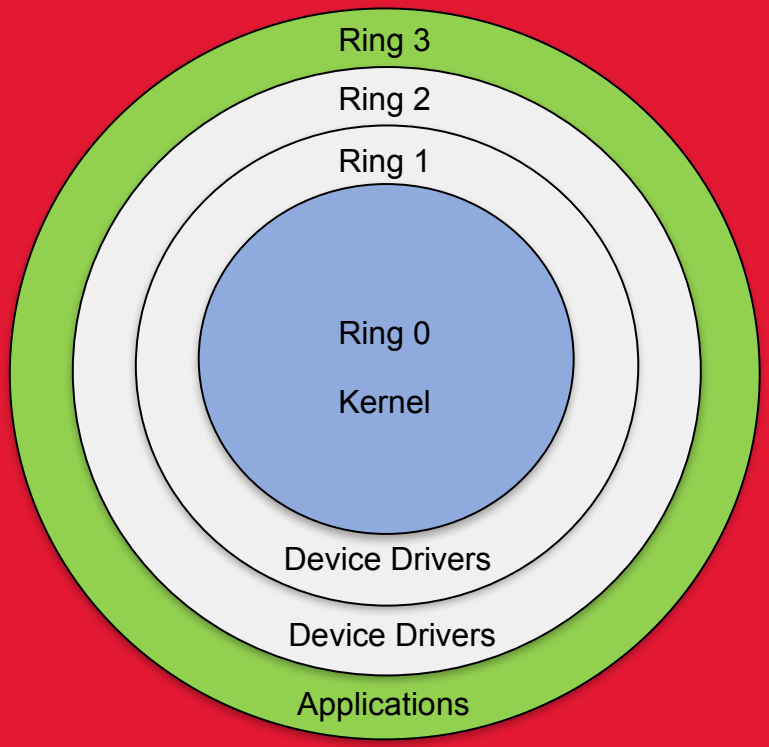


*Only applies to exes flagged as large-address aware

Dual Mode Operation

- Hardware processor with two privilege levels
  - Kernel mode
    - Complete access to the capabilities of the hardware
    - No restrictions
  - User mode
    - Restricted rights
- For Intel processors
  - 4 Privilege levels
    - 0 – 3
    - 0 is kernel mode

Privilege Rings



"Protection Ring", Wikipedia, https://en.wikipedia.org/wiki/Protection_ring

Mode transfer

- User to kernel
  - Interrupts
  - Exceptions
  - System calls
- Kernel to user
  - New process
  - Resume
  - Switch
  - Upcall
- Switching modes is very expensive

Mode transfer

- How does a kernel transition from user mode to kernel mode
  - Two methods
    - Interrupts
      - Asynchronous signal
      - Single processors systems an interrupt causes any processes to stop
      - Multi-processor system – interrupts are taken only on one of the processors
    - Timers

Case Example: Android

- Assigns a unique User ID to each application
  - Runs in a separate process space
- Kernel-level Sandbox
  - Kernel enforces security between applications
  - Cannot interact with each other
  - Limited access to the operating system
- Applies both to native code and operating system apps
- No restrictions on how to write applications
- Memory corruption and arbitrary code execution
- Note: sandbox is *NOT* unbreakable
  - Must compromise the security of the kernel

Process security

- No protection
- Isolation
- Share all or nothing
- Share via access limitation
- Share by capabilities
- Limit use of an object

No protection

- Early versions of windows
- Some embedded environments
- Designed for one user
- No need for isolation, access control, etc.

Process isolation

- Processes unaware of other processes
- Each process: own portion of memory (address space), files, etc.
- OS provides confinement

Isolation

- Processes unaware of other processes
- Each process: own portion of memory (address space), files, etc.
- OS provides confinement
- Example: a word processor, a database and a browser running on a computer
  - All running in different address spaces, to ensure correct operation, security and protection
  - The word processor cannot access the memory of the browser or database.
  - If two processes are running and one of the processes has a problem, that problem should be confined to the process with a problem and not affect the other process.

Isolation

- Processes unaware of other processes
- Each process: own portion of memory (address space), files, etc.
- OS provides confinement
  - What if the OS has a bug?
  - What if the OS is compromised or cannot be trusted?

Isolation

- Virtual machines and cloud computing
- VMWare or Virtual Box
  - multiple operating systems to execute on the same computer without interfering with other program
- Cloud computing
  - Host OS vs. Guest OS
- In this context, each OS is viewed as a process, to be kept separate from the other processes (OSs).
  - Program running in each OS, should not be aware of programs running on another OS, nor should they be aware of the existence of another OS.
  - If a guest OS is compromised, it should not affect the host OS or any other guest OS

Secure processes and threads

- How do we make sure that a process does not interfere with another process or misuse shared resources?
- Must ensure adequate control on access to resources
- Provide rights
- Separate virtual address space
- Execution domain
  - Resources that can be accessed by a process
  - Process should not break the execution domain

Secure Processes and Threads

- In Linux the kernel stores the list of running processes
- Stored in a circular, doubly-linked list called a ‘task list’
- Each element in the ‘task list’ is called a ProcessDescriptor
- ProcessDescriptor contains all info needed for execution
- Add rights to explicitly authorize access to any resource
  - Access to a resource is intercepted and checked for authorization
  - Use quotas to prevent resource denial of service
- Need policy to drive this forward

What About Virtual Address Space?

- Problem:
  - Processes need to be assigned a relatively large VAS to hold code, data, stack, etc.
  - Sharing between address spaces should be convenient
- Solutions?

Solutions for VAS

- One address space per process
- Two address spaces per process
- One address space plus shared address space for OS
- Single level address space

One Address Space Per Process

- Kernel and processes get their own address spaces
- Good isolation
- Some level of protection from the OS
- Simple
- Sharing is more complex

Two Address Spaces Per Process

- Each process gets space for data and code
- Good process isolation
- Protection from the OS
- Data and instructions are separated
- Complex sharing
- Poor address space utilization

One Address Space, Shared OS Space

- OS is shared across processes
- Good process isolation
- Resource sharing
- Security – moderate
- Reduces overall address space per process

Single Level Address Space

- Everything is mapped to this memory space
- Good process isolation
- Logical simplicity
- Uniform protection
- Elegant solution
- Requires large physical address space

Policy

- Secure processes require secure policies
- Must work as a MAC
  - User cannot modify
  - Process cannot “self-modify”
  - Only administrator can modify
- Windows Group Policy
  - Access Control Lists
- Linux
  - SELinux

SELInux

Type Enforcement

- *Object(s):* items in a system that are acted upon (files, IPC, sockets, etc….)
- *Subject(s):* process that are requesting access to an object
- All Objects and Subjects contain a security context
- *Security Context(s)* are composed of four parts
- All Security Context components are checked against the policy to see if access is allowed.
- Type is the base component while role and user are used to further restrict type enforcement

Type Enforcement

- In order to grant access to something, an allow rule must be created, such as:
  - allow user_t bin_t : file {read execute getattr};
    - This means a process with a domain type of user_t can read, execute, or get attributes for a file object with a type of bin_t, there is no significance in the "_t" portion.
    - This rule might be in a policy to allow users to execute shell programs such as the bash shell (/bin/bash).

Security Contexts

system_u:object_r:passwd_exec_t:s0:c0.c2-s2:c0.c1

user:role:type:sensitivity[:category,…][-sensitivity[:category,…]]

TE Access Control

allow user_t bin_t : file {read execute write getattr setattr}

- *Source type(s):* The domain type of the process accessing the object
- *Target type(s):* The type of the object being accessed by the process
- *Object class(es):* The class of object to permit access to
- *Permission(s):* The kind of access permitted for the indicated object class

Domain Transitions

- Analogous to SetUID programs
- Joe running as user_t (untrusted user) needs to change his password. How does Joe change his password?
  - allow user_t passwd_exec_t : file {getattr execute}
  - allow passwd_t passwd_exec_t : file entrypoint
  - allow user_t passwd_t : process transition
- What does this solve? Restricts trusted domain passwd_t and allows user_t to transition to it.
- Implicit domain transitions provided via type_transition.

Users & Roles

- First and second component of a security context
- SELinux usernames and DAC usernames are not synonymous
- Semanage is used to maintain mappings of DAC to SELinux usernames.
- Roles are collections of types geared towards a purpose
- Roles can be used to further restrict actions on the system
- SELinux usernames are granted roles in the system

MLS

- MLS portion of Security Context is composed of 4 parts
  - Low/High
  - Sensitivity/Category
- Includes syntax to define dominance of security levels
- Subjects with range of levels considered *trusted subjects*
- Implements a variation of Bell-La Padula

Using Type Enforcement

- A domain transition is wanted, in order for this to occur, 3 things must exist:
  - The process' new domain type has entrypoint access to an executable file type.
  - The process' current (or old) domain type has execute access to the entry point file type.
  - The process' current domain type has transition access to the new domain type.

Example

- Execute the command “ls -Z /usr/bin/passwd”
  - This will produce the output:

```
-r-s-x-x  root  root system_u:object_r:passwd_exec_t  /usr/bin/passwd
```

- Using this provided information, we can then create TE rules to have a domain transition.
- Three rules are required to give the user the ability to do a domain transition to the password file:
  - allow user_t passwd_exec_t : file {getattr execute};
    - Lets user_t execute an execve() system call on passwd_exec_t
  - allow passwd_t passwd_exec_t : file entrypoint;
    - This rule provides entrypoint access to the passwd_t domain, entrypoint defines which executable files can “enter” a domain.
  - allow user_t passwd_t : process transition;
    - The original type (user_t) must have transition permission to the new type (passwd_t) for the domain transition to be allowed.

Example (cont)

- This isn't very useful by itself since the user would have to specifically say that they want a domain transition. This is where type transition rules are used.
- To create a domain transition by default the following rule is created:
  - type_transition user_t passwd_exec_t : process passwd_t;
  - The type_transition rule indicates that by default on an execve() system call, if the calling process' domain type is user_t and the executable file's type is passwd_exec_t a domain transition to a new domain type (passwd_t) will be attempted
- A type_transition rule causes a domain transition to be attempted by default, but it does not allow it, that's why the other 3 rules had to be created

Example (cont)

- What does this example accomplish?
  - It makes it so that the current user can change the password in the shadow(encrypted password) file, without these rules, this would not be possible even for the root user. The user isn't able to access the file directly though, they must access it through the passwd file so they can't do anything else.

Another Option:

- In addition to Type Enforcement, SELinux also provides a form of role-based access control (RBAC).
  - Roles build on type enforcement to limit the types to which a process may transition based on the role identifier in the process' security context.
  - Using roles is just a way to be even more specific about what access rights are given.

Upcoming

- Next Week: Hardware Security